Governed Agentic Coding (GAC)

Thrash Threshold, Churn, and In-Loop Epistemic Authority Governance for Frontier AI Coding Systems

Governed Agentic Coding reframes agentic development as a governed process that reduces churn within the intrinsic fragility window of long-horizon AI execution.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Frontier AI coding systems operate through extended agentic loops: reading repositories, editing files, running tests, debugging failures, and iterating autonomously. These systems exhibit a repeatable instability pattern. Over sufficiently long loops, declared directives lose binding force. Scope boundaries dissolve, escalation rules are ignored, and the agent begins to self-authorize actions it was explicitly instructed not to take.

This paper defines the **thrash threshold** as the phase transition at which directive authority collapses within an agentic loop. We identify **churn** - predictable semantic fragmentation of the codebase, unnecessary development delays, and increased cost of agent use - as the primary economic consequence of thrash. Loop length under context decay is the dominant predictor of this collapse in current frontier systems.

We introduce **Governed Agentic Coding (GAC)**, a dual-session architecture implementing **Epistemic Authority Governance (EAG)**: a framework that governs when a reasoning system may lawfully continue, escalate, bind, reuse, or terminate. GAC is a concrete EAG instantiation that leverages separation between rule-definition authority (Governor) and execution authority (Coder). We distinguish between prompt-based EAG and structural, in-loop governance. Prompt-based governance improves coherence prior to thrash but still requires user hypervigilance to halt collapse. Structural integration elevates refusal to a first-class success mode, embedding termination semantics directly into the orchestration layer and reducing the need for continuous oversight.

Governed Agentic Coding reframes agentic development as a governed process that reduces churn within the intrinsic fragility window of long-horizon AI execution.

1. Thrash and Its Economic Expression: Churn

Users of frontier AI coding systems consistently observe a pattern:

1. Early iterations are coherent and tightly scoped.
2. Declared directives are respected.
3. Progress is cumulative.
4. After sufficient iterations, constraint adherence degrades.

This degradation manifests as:

- edits outside declared scope,
- debugging despite instructions to escalate,
- reintroduction of removed structures,
- silent invariant mutation,
- abandonment of explicit directives.

We define:

> **Thrash Threshold**: The point in an agentic coding loop at which declared directives cease to exert binding authority over the executor's behavior.

Thrash is governance collapse under context decay.

Its predictable consequence is **churn**.

1.1 Churn Defined

Churn includes:

- semantic fragmentation of the codebase,
- oscillating file changes,
- regression cycles,
- unnecessary debugging passes,
- increased token and compute expenditure,
- schedule delay and cognitive fatigue.

An analogy is a robotic weeder in a strawberry field. When functioning correctly, it removes weeds while preserving strawberries. After crossing the thrash threshold, it loses discrimination and uproots strawberries along with weeds. The result is churned soil rather than cultivated growth.

Thrash is not dramatic failure - it is authority erosion that converts progress into cost.

---

2. Loop Length and Context Decay

Let an agentic coding loop consist of repeated cycles:

$$Cycle_k = (Read, Propose, Edit, Execute, Evaluate)$$

In current frontier systems:

- Governance exists as contextual instruction.
- Session memory is finite.
- Context is summarized, compacted, or evicted.
- Execution reasoning competes with constraint salience.

As loop length increases, directive persistence weakens.

Short loops remain stable. Long loops drift.

Restating constraints improves adherence but does not eliminate fragility. Governance that exists only in prompt context is not structurally persistent.

Loop length remains an intrinsic destabilizing factor.

---

3. Prompt-Based EAG vs Structural Governance

3.1 Prompt-Based Epistemic Authority Governance

When users implement EAG via prompt engineering:

- Invariants are declared.
- Scope boundaries are specified.
- Refusal and escalation rules are described.
- Sessions are intentionally bounded.

This produces observable improvements:

- fewer scope violations,
- reduced unsolicited debugging cascades,
- lower churn before thrash onset,
- improved coherence per loop segment.

However:

- Governance remains context-bound.
- Constraint salience decays under long loops.
- The user must detect thrash and intervene.
- Refusal is not structurally enforced.

Prompt-based EAG improves stability but still depends on user vigilance.

3.2 Structural In-Loop Governance

Structural governance embeds authority separation into orchestration:

- Constraints persist outside volatile context.
- Executor cannot self-amend constraints.
- Escalation precedes mutation.
- Refusal/halt is a first-class outcome.
- Termination is treated as success, not failure.

Thrash becomes structurally bounded rather than user-monitored.

---

4. Dual-Session Architecture (GAC)

Governed Agentic Coding partitions authority:

4.1 Governor Session

The Governor:

- defines invariants,
- establishes scope,
- specifies refusal/halt conditions,
- adjudicates constraint mutation.

The Governor does not mutate repository state.

4.2 Coder (Executor) Session

The Coder:

- reads and edits files,
- runs tests,
- operates within declared constraints.

The Coder cannot redefine constraints.

Authority separation prevents executor self-amendment.

---

5. Escalation and Refusal as Success

Let:

- C be active constraints.
- A be a proposed action.

If:

$$\exists c \in C \text{ such that } c(A) = DENY$$

then:

$$Execute(A) = \emptyset$$

and escalation is mandatory.

In structural governance:

Refusal is success. Halting preserves invariants. User hypervigilance is no longer required.

---

6. ChatGPT–Claude Pairings and Architect-Style Review

Pairing large language models - such as ChatGPT and Claude - is now common practice. Users often separate roles informally:

- one model drafts or executes,
- the other reviews, critiques, or restructures,
- iterations pass between models for refinement.

This dual-model pattern resembles governance intuitively. It introduces cross-checking and improves output quality relative to single-agent loops.

However, in most ChatGPT–Claude pairings:

- Governance remains prompt-based.
- Constraints are described, not enforced.
- Escalation semantics are advisory.
- Executor autonomy persists within each loop.
- Loop-length fragility remains unchanged.

The models may alternate roles, but neither is structurally prevented from self-amendment during its execution phase. Directive authority remains context-bound.

Similarly, in environments such as Replit, an "Architect" function may review or critique code after execution. While beneficial, this is a post-hoc corrective

layer. It does not govern the executor loop itself. Mutation occurs first; critique follows.

In contrast, GAC requires:

- Governance active during execution.
- Escalation before mutation.
- Refusal as an orchestration-level outcome.
- Non-self-amendment of constraints.

ChatGPT–Claude pairings demonstrate that practitioners intuitively seek separation of roles. GAC formalizes this instinct and installs authority partition structurally rather than conversationally.

---

7. Replay and Lineage as Extensions

Replay and lineage tracking can reinforce governance continuity by:

- preserving invariant history,
- supplying structured context,
- reducing accidental constraint drift.

These mechanisms enhance stability but are not prerequisites for EAG benefits.

---

8. Broader Context

Meaning-producing systems require separation between:

- rule-definition authority, and
- execution authority.

Frontier agentic coding systems fuse these roles within a single volatile loop. As loop length increases, directive authority decays and churn emerges.

Thrash is not model incompetence. It is authority fusion under context constraints.

Installing epistemic authority governance in-loop is a step toward stable, economically sustainable agentic orchestration.

---

9. Conclusion

Thrash in frontier AI coding systems is the collapse of directive authority under long-horizon execution. Its consequence is churn: fragmentation, cost inflation, and delay.

Prompt-based EAG reduces churn but still depends on vigilant intervention. Structural integration elevates refusal to a first-class success mode and embeds governance directly into orchestration.

Governed Agentic Coding:

- partitions authority,
- enforces escalation before mutation,
- treats halting as preservation,
- reduces churn within bounded execution horizons.

True stability requires governance installed structurally within the agentic loop - not merely remembered in prompts.

Corpus-Wide Reference Positioning

This paper argues that frontier AI coding systems operate through extended agentic loops: reading repositories, editing files, running tests, debugging failures, and iterating autonomously. Its nearest external comparison fields are software, governance, ai. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. Retrieval, tool use, and policy layers can reduce unsupported output without eliminating model error, source error, omission, bias, or the environmental and institutional costs of scale. These

limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E2**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E3**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1.* NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

[**E4**] Reproducible Builds Project. *Formal Definition of Reproducible Builds.* reproducible-builds.org source **Relationship:** Adjacent reproducibility practice; byte reproduction is compared with, but not equated to, semantic or constitutional continuity.

[**E5**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E6**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority;

included to expose differences between political and computational constitutionalism.

[**E7**] Lawrence Lessig. *Code: Version 2.0*. Basic Books, 2006. https://codev2.cc/
Relationship: Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

Ableman corpus

[**A1**] Adam Ableman. *Sovereign Software*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-24C6159B98023A9C`;
source SHA-256
`6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems*. Ableman Corpus current edition; work identity `PAPER.PUB.20472196`;
source SHA-256
`3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Semantic Fragmentation and Constitutional Structure in Software Systems*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-B307F65B51C503DE`;
source SHA-256
`9D2A9D79DE8BC3F2F8ABECA073E4D3607F7623F4368E63F8036081E39B4C75C3`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Governed Coding and Debugging*. Ableman Corpus current edition; work identity `PAPER.PUB.18115533`;
source SHA-256
`2469EDA96DBC1E207855A9AD27314A7FAB59FA35E6EFEF9288C30F949376BF16`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Constitutional Operating Systems*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-F5FB8FA82270B989`;
source SHA-256
`E982E4D6BD4B519EED6A7438367844954D6D5F1C2C4CC0099AAAC4AB9A00D3AB`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Ableman Constitutional Governance*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-79353AF163A4B0F1`;
source SHA-256
`FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Theory*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D2D97C4042FD281F`;
source SHA-256
`FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.18629666